5 Bridging from Kalman to Neuron

This linearity assumption is a serious one. Its justification lies in the fact that the patterns produced in the early stages when it is valid may be expected to have strong qualitative similarity to those prevailing in the alter stages when it is not.
—Alan M. Turing, 1952 [Ala52]

5.1 Introduction

Richard Feynmann suggested that the goal of physics was to create mathematical models that explain observations [Fey65]. Observations are data. Our goal in this book is to use models of neurons and brains to fit to brain data. We will do this in order to observe and track brain activity. And we seek to use this observability to develop control algorithms for neural activity.

So our first mission is to determine: Can we fit models to brain data?

The problem is not whether we can insert data into a model. Mindless computation is an occupational hazard of computational neuroscience.

The problem is that neurons, and brains, are not linear systems. And the problem with nonlinear systems is that they can do things that linear systems never do, such as not having any truly periodic behaviors. If your brain is generating sine waves, you are in trouble.

By the mid-twentieth century, it had become apparent that even simple nonlinear systems might generate deterministic yet aperiodic behavior [Lor63]. Such nonlinear systems might have no stable states. Instead, they were characterized by a plethora of unstable states, each with a different frequency. Another hallmark of such systems was sensitivity to initial conditions. If the system were in one of two nearby states, the future evolution might exponentially diverge. This extreme form along the spectrum of the dynamics of nonlinear systems would be termed *chaos* [LY75]—dissipative, bounded, uniformly unstable with sensitivity to perturbation, and aperiodic behavior that is anything but random.

Why bring up chaos in a book about the nervous system? It is not at all clear that brains, or even simple nervous system networks, generate behaviors that we can classify as chaotic.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

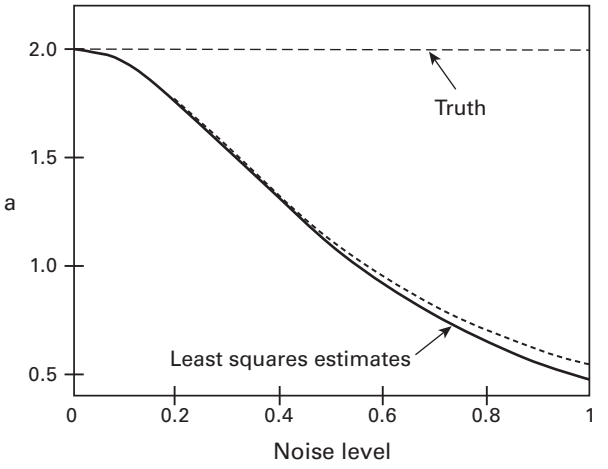


Figure 5.1
Demonstration of least squares failure to fit a simple nonlinear model. The true parameter $a = 2$ is severely underestimated whether added noise is uniform (dotted line) or Gaussian (solid line). Reproduced with permission from [MS04].

I say this not because there does not exist a huge literature arguing that brains do.[1] It is because we have almost universally failed to demonstrate that our analysis of brain states is of true states—where the recurrence of similar states led to similar subsequent events. The same can be true of the atmosphere and oceans, where the lack of a true *analog* [Lor63], a state repeating a previous state, leaves the ultimate issue of whether such complex physical systems are chaotic or not an infinitely open question.

But finding a brain state analog is not the point. The point is that brains are made up of floridly nonlinear elements. Neurons have membranes that generate action potentials that can be modeled with the Hodgkin-Huxley or Fitzhugh-Nagumo equations. Their synaptic connections are nonlinear. Their ion concentration dynamics are nonlinear, as will be discussed later (chapter 12). In general, we are pretty safe in assuming that any method to fit models to brain data using linear models might not be the optimal choice.

And the problem with fitting models to nonlinear systems is that it is notoriously hard to do. If there is noise (When is there not noise in a system and in its measurements?), a system that is sensitive to small perturbations causes trouble. A beautiful example is offered by McSharry and Smith [MS99a]. They started with a *logistic map*. This is a simple equation that has long had a role in modeling in population biology.

$$x_{i+1} = ax_i(1 - x_i) \tag{5.1}$$

1. The author has, for better or worse, contributed to some of this controversy.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

For suitable values of the parameter a, this almost trivially nonlinear map generates chaotic trajectories. And using Gauss's least squares method from chapter 1, and knowing the model that generated the data, gives the *wrong* value of a—dramatically wrong, as figure 5.1 shows. One can find that fitting a cost function to determine such parameters might well reject the very model that generated the data [MS04].

Our goal is a bit less severe than this. *We are not out to reengineer the brain*. That is, it would be splendid to have a model that truly represented all of the brain's components, or even a single neuron. But no such model exists. And if it did, it would be so complex that, were you to have a computer large enough to run the model fitting for some fraction of eternity, you still would not get the correct parameters. There would be too many parameters to fit any of them with much certainty. Our goal is to fit dynamics. Observe the dynamics of the nervous system, and you can potentially control it based on that tracking.

So let's track nonlinear dynamics.

5.2 Variables and Parameters

We are interested in this book in tracking natural systems using autonomous systems of differential equations. Autonomous systems have time only present in the derivatives, as follows:

$$\dot{x}_1(t) = f_1(x_1(t), x_2(t), \ldots x_n(t), \lambda_1, \ldots, \lambda_m)$$

$$\dot{x}_2(t) = f_2(x_1(t), x_2(t), \ldots x_n(t), \lambda_1, \ldots, \lambda_m)$$

$$\vdots$$

$$\dot{x}_n(t) = f_n(x_1(t), x_2(t), \ldots x_n(t), \lambda_1, \ldots, \lambda_m)$$

where we allow for n variables x, m parameters λ, and $\dot{x}$ indicates derivative with respect to time.

Assuming vectors in the notation, one would write

$$\dot{x} = F(x, \lambda) \tag{5.2}$$

for the model, leaving off noise for now. We assume, as before, a measurement function

$$y = A(x) + r \tag{5.3}$$

where r is random noise.

If the measurements are a vector of values y, then our goal is to find the highest probability, P, of observing these data y, given the model x and parameters λ such that $P(y|x, \lambda)$ is maximized. This probability peak yields the parameters that are most likely to have accounted for the data observed—the *maximum likelihood*.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

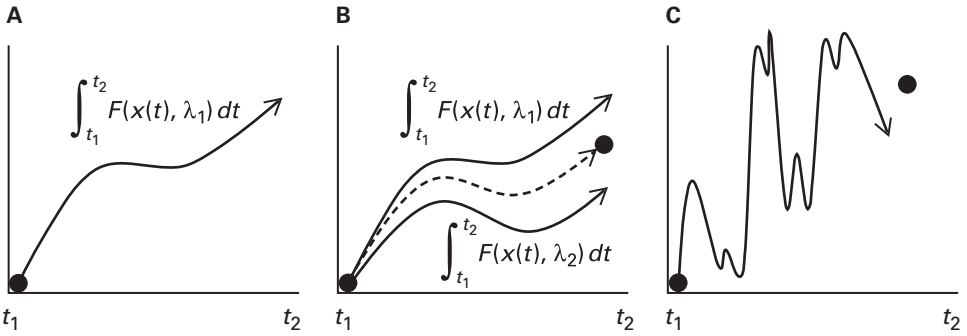


Figure 5.2
Direct integration of a differential equation using an initial value boundary condition shown as a filled circle in A. In B, a second boundary condition is known, a true value that the underlying system produced, and one can iterate the integration of $F(x(t), \lambda_i)$ to hit the target and optimize the parameters λ_i. C shows a chaotic trajectory where sensitivity to initial conditions renders a single shooting effort almost impossible to optimize.

The situation is complicated if there is uncertainty in the variables as well as in the parameters. The *errors-in-variables* problem has been well studied (see reviews in [VTK04, Sod07]). Throughout this book, we will need to estimate both the variables and parameters in our model fitting to data—the *dual-estimation* problem. It will turn out, later, that an ensemble Kalman filter is ideal for such dual-estimation problems through the use of an *augmented state*, which includes both variables and parameters to be estimated. But we are getting ahead of ourselves.

For now let's first assume that you actually know the model that generated the dynamics to be tracked, and that there is no uncertainty in the independent variables. Even with this advantage, for noisy nonlinear systems, fitting the model to the data is not easy.

One way to do this is to start with an initial condition, x_0, and integrate the model forward from time t_1 to a final time t_2. This is a single boundary value problem. If we know the model (differential equation), and there is no noise, then the solution is exact. The situation is illustrated schematically in figure 5.2A. We did this, for instance, when we integrated the Hodgkin-Huxley or Fitzhugh-Nagumo equations in previous chapters. Are the trajectories of these integrations valid? Well, valid for what? They are trajectories from numerical integration. Do they reflect anything from nature? Doubtfully. Do they reflect anything from even those models, if we did not know the parameters of the models (e.g., the maximal conductances for sodium, potassium, and leak in the Hodgkin-Huxley equations)? No. And if there was noise in the models, but you did not know the noise at each point in time, would your trajectory happen to be the the same as the model's? Of course not. This is the solution of the *direct problem* (as opposed to the *inverse problem*, where you are given data and want to calculate the most likely model trajectory underlying the measurements).

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

So the next more complex thing to do is to introduce a two-point boundary value problem. You have initial conditions, but you also know the final state. You want to fit the parameters to the model so that the solution is consistent with the endpoint. The first boundary condition is initial—you aim the equations. The trajectory then shoots, and your final value is compared with the value of the system. So you adjust the *aim* of the equations by resetting the parameters and try again. This *shooting method* is schematized in figure 5.2B. For fairly well-behaved equations, shooting methods are very good means of picking the best set of parameters λ_i. An excellent discussion of shooting methods can be found in [Pre07].

But what if nonlinearities in the dynamics increase as schematized in figure 5.2C? Chaotic dynamical systems shred information as they evolve, so that any sets of initial conditions and parameters close to the true solution will wildly diverge after a relatively short time. Even if you had the exact initial condition and parameter values, if you introduce noise you are again on the path to exponential divergence from the truth. In fact, using computers introduces trouble—the roundoff error of digital precision will create errors of the size of the system being simulated after a certain number of iterations. This is the *shadowing problem*: You want to know if any of your calculated trajectories remain close to the true trajectory [SGY97]. The bottom line is that you can shoot for a solution, but not for very long (not long in comparison with the instability in the equation).

So if you cannot shoot for a relatively long time in a chaotic system and come close to the true solution, a logical choice is to shorten the shooting range. *Multiple shooting* improves tracking by shooting over shorter distances, and repeating the process in small saltatory steps. These methods are well developed, and one can measure data and use multiple shooting to solve the inverse problem for nonlinear systems. Nevertheless, multiple shooting methods for chaotic systems are well known to be computationally expensive to implement, difficult to tune, and restricted to models that are differentiable [SSKV02].

5.3 Tracking the Lorenz System

Since Edward Lorenz wrote his paper entitled *Deterministic Nonperiodic Flow* in 1963 [Lor63], his model has been a paradigmatic system of nonlinear dynamics. Why introduce a highly simplified model of atmospheric convection at this point? Much of the literature that feeds the developments to follow in this and subsequent chapters come from physics and meteorological literature where the cutting-edge research in system tracking, observation, and prediction often takes place. It often appears as if no geophysicist or numerical meteorologist can write a paper without thinking about using the 1963 Lorenz model.[2] So we will take some time to describe it here, as we will use it later.

2. There is a more complex model that Lorenz published in 1998 [LA98], which we will not use in this book.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

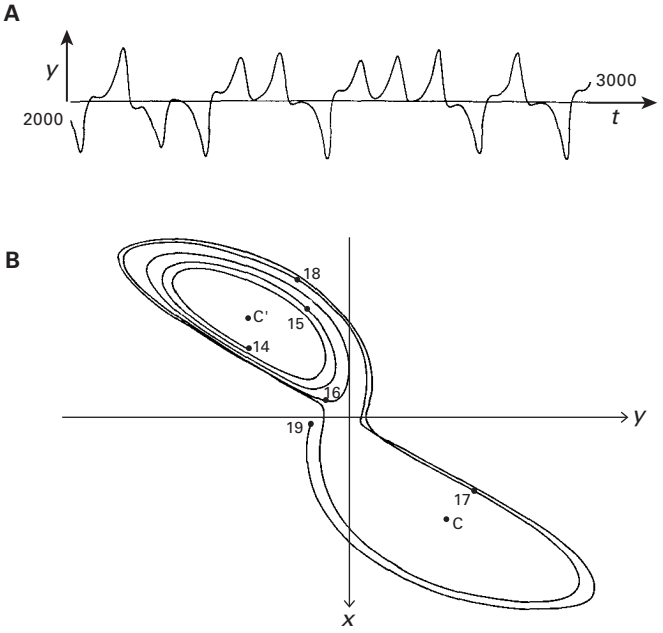


Figure 5.3
Solutions of Lorenz equations shown in A for y versus time, after skipping initial transient, and B shows plot of y versus x. Reproduced from [Lor63] with permission.

The differential equations of the Lorenz model are

$$\begin{aligned} \dot{x} &= -\sigma x + \sigma y \\ \dot{y} &= -xz + rx - y \\ \dot{z} &= xy - bz \end{aligned} \tag{5.4}$$

The variables in this set of equations refer to convection of fluid in an experiment where fluid is heated from below and convection rolls develop. The variable x refers to the velocity of convective flow, y the temperature difference between ascending and descending fluid, and z the distortion of the vertical temperature profile.[3] The parameter σ refers to the Prandtl number, r the Raleigh (Reynolds) number, and b is an adjustable parameter. For values of these parameters $\sigma = 10$ and b $= 8/3$, Lorenz found that values of $r > 24.74$ gave chaotic solutions (figure 5.3).

3. Although Lorenz's original paper [Lor63] is readable on the meaning of these variables, it is a very technical paper. An excellent overall description of these dynamics can be found in [ASY97].

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

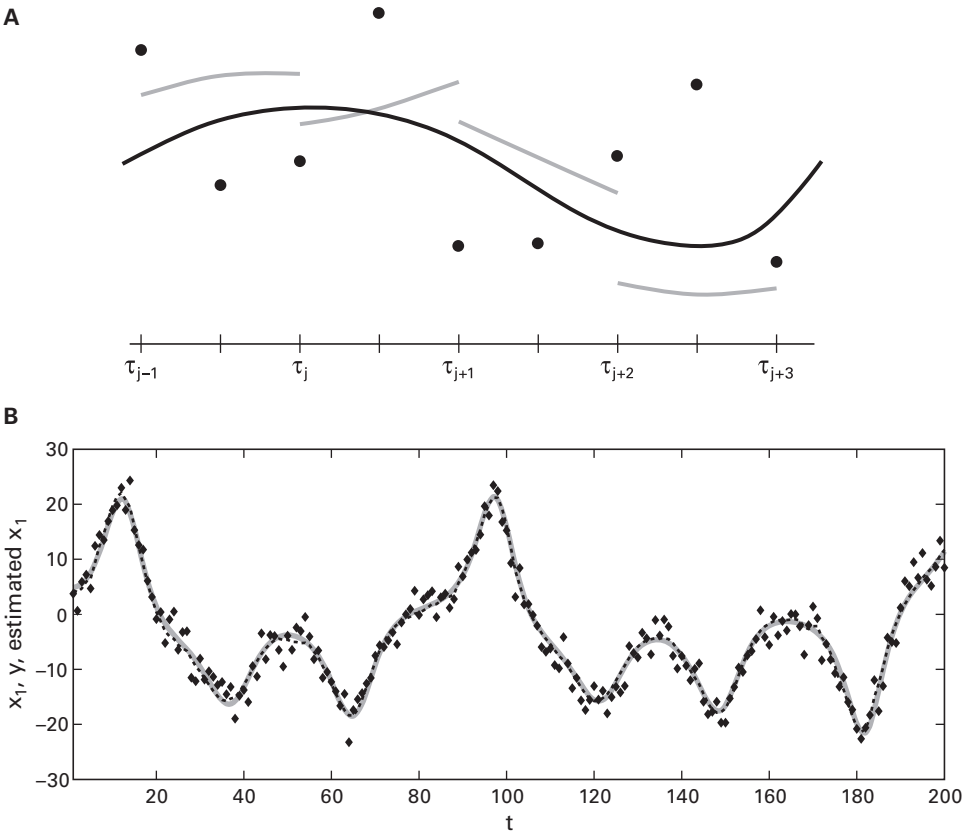


Figure 5.4 (plate 1)
Multiple shooting schematic for solving inverse parameter estimation shown in A, and an actual example of noisy Lorenz system is shown in B. Reproduced from [VTK04] with permission.

Let's assume that you knew you were observing data from a Lorenz system, and you had the equations. You are given noise-contaminated measurements of the x variable only, and you do not know the parameters. Voss and colleagues have examined the problem in detail [VTK04]. First, they start with an initial condition and known parameters and generate a true trajectory. It is important here to remember that these parameters are fixed. Then they measure x with noise mixed in, and ask whether they can solve for the parameters—an inverse problem. Figure 5.4A (plate 1) shows a schematic of multiple shooting to solve this inverse problem. Figure 5.4B shows their actual multiple shooting fit to these data after three passes over the data. Following twenty iterations over the full data set, the fit will be excellent, and their parameter estimates are close to the actual parameters, within 2.5 to 3.4%. This example has nothing about real-time use.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

Now repeat this measurement and estimation scenario using the unscented Kalman filter. We use the same noisy x measurements and let the parameters be unknown, first forming an *augmented state vector* by adding the parameters to the state variables. This forms a six-dimensional vector for the Lorenz system of equations (5.4). We will use augmented state vectors throughout this book in dealing with neuronal systems.

Voss and colleagues [VTK04] make a first pass over the data with a known bad choice of parameters, and use the final parameter values to make another pass at fitting the data, as illustrated in figure 5.5 (plate 2). Their parameter estimates are now superb, within 0.3 to 1% error. Although they did not use the unscented Kalman filter in a real-time setting, it could have been used as such.

5.4 Parameter Tracking

So far we have discussed parameter estimation. There are no dynamics involved, and we (and the citations we have referred to) have focused on parameters that must be found. Once found, they become system constants.

Nothing is constant in the nervous system. Since Sherrington's classic text analyzing the dynamics of reflex variability [She06], spinal reflexes, nerve propagation through branch points, synaptic probabilistic release, basal ganglia rhythmicity, cortical rhythms—nothing ever seems to stay constant.[4] So it makes a great deal of sense to let the parameters change in our efforts to computationally track neural dynamics.

In *parameter tracking*, one assumes that the parameters, like the variables, change their values as a function of time. What is the difference? Implicit in the parameter tracking formulations is that the time scale of the rate of change of parameters is much slower than the changes in variables. One can then apply trivial dynamics to parameters in the sense that

$$\lambda(t+1) \approx \lambda(t)$$

In the recursive formulation of the Kalman filter, however, fitting the augmented state vector with such trivial dynamics on the propagation of the parameters still permits the resetting of the parameters to adapt as the data is fit with each step of the recursive algorithm. Recall that we employed static models in our examples in the exercises in chapter 2.[5]

In implementing this strategy, one needs to add noise to the parameters in order to ensure that the algorithm will *search* about for better parameters with each step [VTK04]

$$\lambda(t+1) = \lambda(t) + r(t)$$

where $r(t)$ is drawn from a random distribution or estimated from the expected variance.

4. An interesting facet of such variability has been seen in the use (and misuse) of reflex measurements in the treatment of spasticity [WS93].

5. A nice discussion of the use of trivial dynamics for static models in Kalman filtering is also found in [Str86].

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

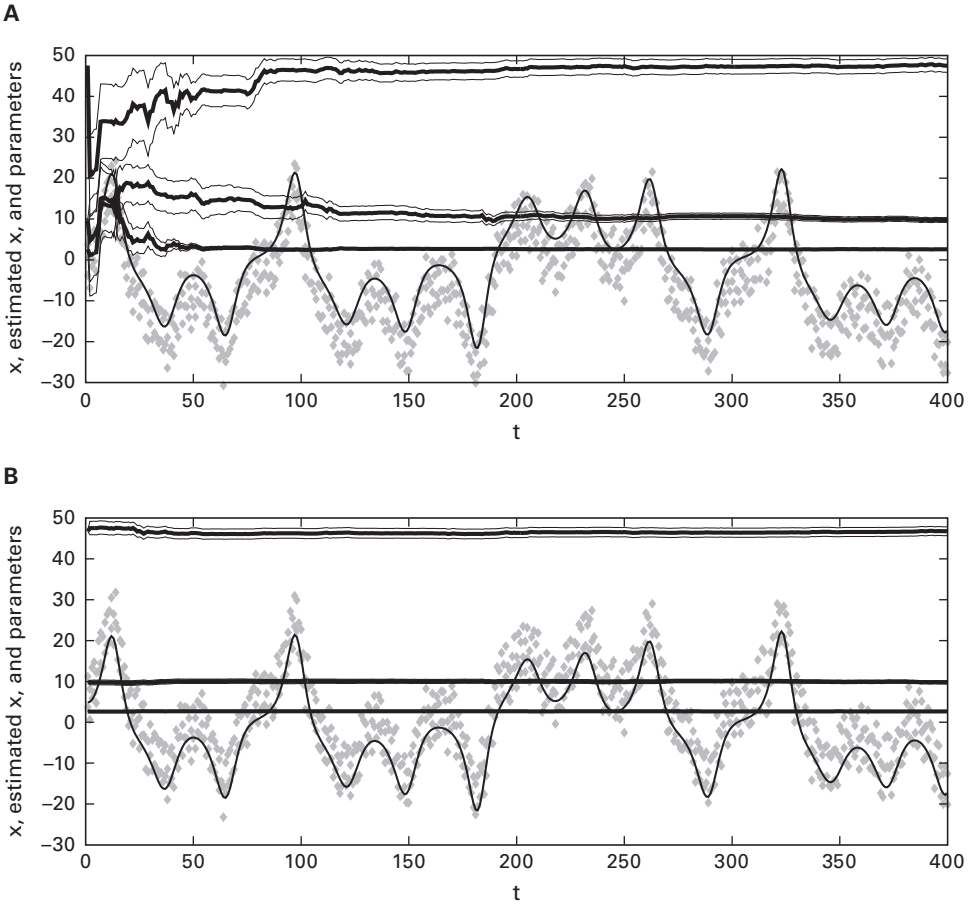


Figure 5.5 (plate 2)
A shows the first pass of the unscented Kalman filter through the noisy x output (dots) of the Lorenz system. The three parameters being estimated are shown as solid lines, with thin line confidence limits gradually converging from left to right. The values at the right are then used to initiate another pass at parameter estimation, shown in the lower panel, B. Although parameter estimation improves a bit more, the upper panel demonstrates the potential power of this technique to work with one pass in real-time applications. Modified from [VTK04] with permission.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

5.5 The Fitzhugh-Nagumo Equations

The bridge to neuroscience from this world of nonlinear control theory appears to occur in five paragraphs toward the end of the tutorial paper by Voss and colleagues [VTK04]. In prior work, they had explored unscented Kalman filtering of the stochastic Van der Pol equations [SSKV02]. But no neuroscientist (certainly not the author) would have recognized the nervous possibilities at a reading of that earlier article. They chose the Fitzhugh-Nagumo equations (4.3), and set up the following scenario. They assumed that all one could do was to measure voltage, x. They added considerable amounts of noise to the measurement and set parameters for this equation at the same levels we used in exercise 4.2: $a = 0.7$, $b = 0.8$, $c = 3$. But they allowed the applied current, I, in equations (4.3) to float and become a function of time, $I(t)$ with the trivial dynamics of a parameter.

In generating their model, Voss and colleagues [VTK04] applied a nontrivial dynamics to $I(t)$—a sequence of connected cosine half waves as shown in figure 5.6 (plate 3). In the figure, a constant noise variance was applied to the measured parameter $I(t)$ with each time step. The initial values of $I(t)$, and the unobserved variable, y, were just set to zero, as shown in panel B of the figure. The initial value for the variable x, the voltage, was set to the actual value of x used in the model simulation.[6]

The results in figure 5.6 are remarkable. There is no analog to accomplish this using previous techniques such as multiple shooting (although in all fairness, the author is unaware of comparable alternative efforts).

The importance of these results is that it demonstrates in a simplified neuronal model that the estimation of inaccessible parameters, and tracking of unknown parameters, is possible in the case of neuronal dynamics. It is the starting point for our explorations of using ensemble Kalman filtering techniques on data from single cells, reconstructing cellular networks, tracking cortical dynamics, Parkinson's disease control, and the other scenarios to be explored in later chapters. Nevertheless, we have so far discussed no model capable of accounting for dynamics more extensive than a piece of neuronal membrane. To work with the nervous system, we need to be able to account for spatially extended systems. At the end of [VTK04], the authors suggested that *it should also be possible to perform some kind of parameter estimation for spatiotemporal systems*. In the next chapter, we explore how to begin to accomplish this for cortical dynamics.

6. The authors [VTK04] graciously made available their insightful code structure with publication of the article. Without this generous offer, many of the developments to be derived later in this book would have been much delayed and not performed as well. It is a strong argument for the publication of code as supplementary material with all computational papers. Otherwise, one of the key aims of scientific publication, a sufficient description of methods such that others can replicate the work, can be significantly impaired. It's easy to write papers that no one can replicate.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

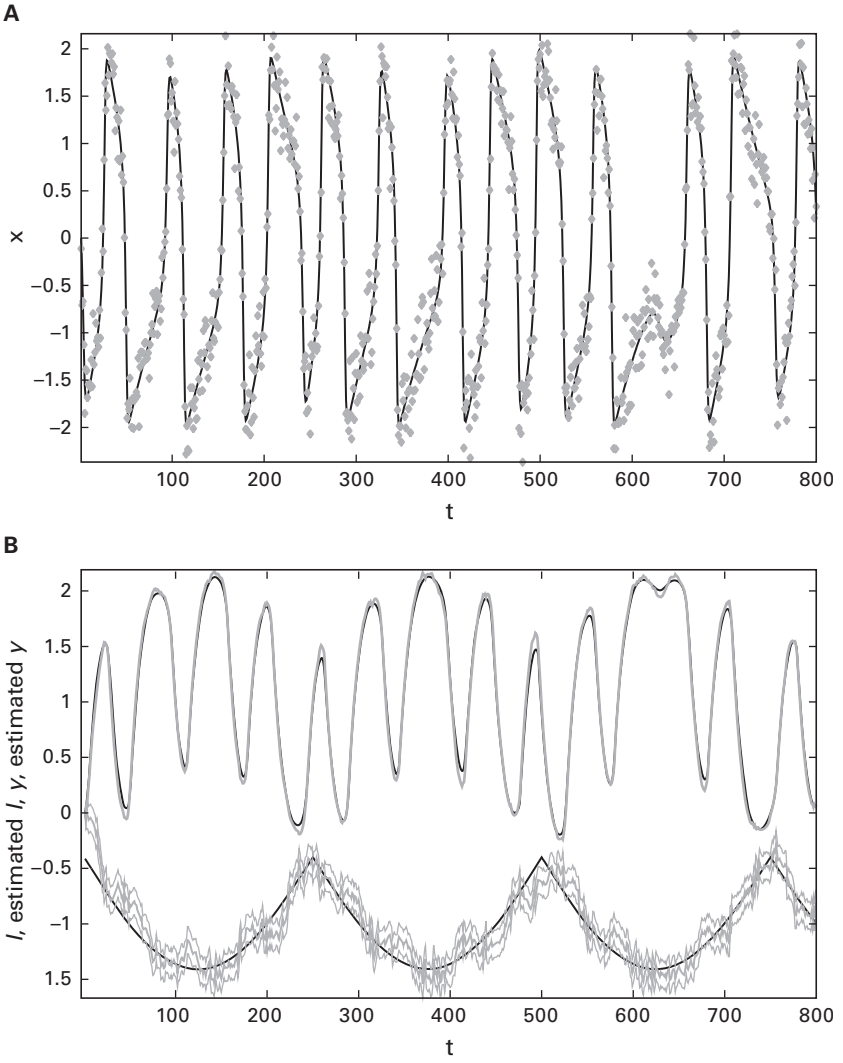


Figure 5.6 (plate 3)
Variable and parameter tracking in the Fitzhugh-Nagumo equations. A shows the underlying trajectory (solid line), the noisy measurements (dots), and B shows the reconstruction (red) of the unmeasured variable y (black, nearly overlapping) in the upper half of the panel, and the applied current $I(t)$ and its estimation as a tracked parameter (magenta) with 1 standard deviation confidence limits (thin magenta lines). Reproduced from [VTK04] with permission.

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

Exercises

Here we will explore the Voss et al. [VTK04] algorithmic structure in detail. Although we will make many modifications to their original algorithm throughout this book, we have not found a better overall subroutine structure than the one presented in their original calculations. What we present below is both Matlab and Octave compatible.

There are five scripts in this algorithm, which we denote as `voss.m`, `vossFNfct.m`, `vossFNint.m`, `vossobsfct.m`, and `vossut.m`. Four are callable subroutines. The main program, `voss.m`, is:

```
% Algorighm originally made available by Henning U. Voss, 2002
% "Nonlinear dynamical system identification
%  from uncertain and indirect measurements"
% HU Voss, J Timmer, J Kurths - International Journal
%  of Bifurcation and Chaos, Vol. 14, No. 6 (2004) 1905-1933
% Reproduces Figure 9 from paper
% Modifications by S. Schiff 2009 - Matlab and Octave Compatible
clear all; close all;
  global dT dt nn % Sampling time step as global variable
  dq=1; dx=dq+2; dy=1;
        % Dimensions: (dq for param. vector, dx augmented state, dy observation)
  fct='vossFNfct'; % this is the model function F(x) used in filtering
  obsfct='vossFNobsfct';  % this is the observation function G(x)
  N=800; % number of data samples
  dT=0.2; % sampling time step (global variable)
  dt=dT; nn=fix(dT/dt); % the integration time step can be smaller than dT
% Preallocate arrays
  x0=zeros(2,N);      % Preallocates x0, the underlying true trajectory
  xhat=zeros(dx,N);   % Preallocate estimated x
  Pxx=zeros(dx,dx,N); % Prallocate Covariance in x
  errors=zeros(dx,N); % Preallocate errors
  Ks=zeros(dx,dy,N);  % Preallocate Kalman gains
% Initial Conditions
  x0(:,1)=[0; 0]; % initial value for x0
% External input current, estimated as parameter p later on:
  z=[1:N]/250*2*pi; z=-.4-1.01*abs(sin(z/2));
% RuKu integrator of 4th order:
for n=1:N-1;
  xx=x0(:,n);
  for i=1:nn
    k1=dt*vossFNint(xx,z(n));
    k2=dt*vossFNint(xx+k1/2,z(n));
    k3=dt*vossFNint(xx+k2/2,z(n));
    k4=dt*vossFNint(xx+k3,z(n));
    xx=xx+k1/6+k2/3+k3/3+k4/6;
  end;
```

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

```
  x0(:,n+1)=xx;
end;
x=[z; x0]; % augmented state vector (notation a bit different to paper)
xhat(:,1)=x(:,1); % first guess of x_1 set to observation
% Covariances
  Q=.015; % process noise covariance matrix
  R=.2^2*var(vossFNobsfct(x))*eye(dy,dy);
        % observation noise covariance matrix
  randn('state',0);
  y=feval(obsfct,x)+sqrtm(R)*randn(dy,N); % noisy data
  Pxx(:,:,1)=blkdiag(Q,R,R);% Initial Condition for Pxx
% Main loop for recursive estimation
for k=2:N
  [xhat(:,k),Pxx(:,:,k),Ks(:,:,k)]=...
        vossut(xhat(:,k-1),Pxx(:,:,k-1),y(:,k),fct,obsfct,dq,dx,dy,R);
  Pxx(1,1,k)=Q;
  errors(:,k)=sqrt(diag(Pxx(:,:,k)));
end; % k
% Results
  chisq=...
    mean((x(1,:)-xhat(1,:)).^2+(x(2,:)-xhat(2,:)).^2+(x(3,:)-xhat(3,:)).^2)
  est=xhat(1:dq,N)'; % last estimate
  error=errors(1:dq,N)'; % last error
  meanest=mean(xhat(1:dq,:)')
  meanerror=mean(errors(1:dq,:)')
% Plot Results
subplot(2,1,1)
  plot(y,'bd','MarkerEdgeColor','blue', 'MarkerFaceColor','blue',...
         'MarkerSize',3);
  hold on;
  plot(x(dq+1,:),'k','LineWidth',2);
  xlabel('t');
  ylabel('x_1, y');
  hold off;
  axis tight
  title('(a)')
subplot(2,1,2)
  plot(x(dq+2,:),'k','LineWidth',2);
  hold on
  plot(xhat(dq+2,:),'r','LineWidth',2);
  plot(x(1,:),'k','LineWidth',2);
  for i=1:dq; plot(xhat(i,:),'m','LineWidth',2); end;
  for i=1:dq; plot(xhat(i,:)+errors(i,:),'m'); end;
  for i=1:dq; plot(xhat(i,:)-errors(i,:),'m'); end;
  xlabel('t');
  ylabel('z, estimated z, x_2, estimated x_2');
  hold off
```

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

```
  axis tight
  title('(b)')
```

The script that performs the unscented transform calculations is `vossut.m`:

```
% Unscented transformation from Voss et al 2004
% This Function has been modified by S. Schiff and T. Sauer 2008
function [xhat,Pxx,K]=vossut(xhat,Pxx,y,fct,obsfct,dq,dx,dy,R);
N=2*dx;      %Number of Sigma Points

Pxx=(Pxx+Pxx')/2; %Symmetrize Pxx - good numerical safety
xsigma=chol( dx*Pxx )'; % Cholesky decomposition - note that Pxx=chol'*chol

Xa=xhat*ones(1,N)+[xsigma, -xsigma]; %Generate Sigma Points
X=feval(fct,dq,Xa); %Calculate all of the X's at once

xtilde=sum(X')'/N; %Mean of X's
X1=X-xtilde*ones(1,size(X,2)); % subtract mean from X columns
Pxx=X1*X1'/N;
Pxx=(Pxx+Pxx')/2; %Pxx covariance calculation

Y=feval(obsfct,X);
ytilde=sum(Y')'/N;
Y1=Y-ytilde*ones(1,size(Y,2)); % subtract mean from Y columns
Pyy=Y1*Y1'/N + R; %Pyy covariance calculation

Pxy=X1*Y1'/N; %cross-covariance calculation

K=Pxy*inv(Pyy);
xhat=xtilde+K*(y-ytilde);
Pxx=Pxx-K*Pxy'; Pxx=(Pxx+Pxx')/2;
```

And three small callable functions round out this algorithm in `vossFNint.m`:

```
%This function calculates the Fitzhugh-Nagumo equations
function r=vossFNint(x,z)
a=.7; b=.8; c=3.;
r=[c*(x(2)+x(1)-x(1)^3/3+z); -(x(1)-a+b*x(2))/c];
```

`vossFNobsfct.m`:

```
%This function strips out and returns just the observation variable, voltage
function r=vossFNobsfct(x)
r=x(2,:);
```

and `vossFNfct.m`:

```
% RuKu integrator for FitzHugh system with parameters
% Modified from Voss et al 2004.
function r=vossFNfct(dq,x)
```

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

```
global dT dt nn
p=x(1:dq,:);
xnl=x(dq+1:size(x(:,1)),:);
for n=1:nn
  k1=dt*fc(xnl,p);
  k2=dt*fc(xnl+k1/2,p);
  k3=dt*fc(xnl+k2/2,p);
  k4=dt*fc(xnl+k3,p);
  xnl=xnl+k1/6+k2/3+k3/3+k4/6;
end
r=[x(1:dq,:); xnl];

function r=fc(x,p);
a=.7; b=.8; c=3.;
r=[c*(x(2,:)+x(1,:)-x(1,:).^3/3+p); -(x(1,:)-a+b*x(2,:))/c];
```

5.1. For the first exercise, let's modify this program so that it tracks all parameters, a, b, and c, in addition to the input current.

• *Hint*: The original integration to get x0 is unchanged. You need to increase the value of dq as you add parameters. I suggest augmenting the state vector and estimating the additional parameters one at a time. Start with a. Once this is working, add b and c. You need to modify the setting of the covariance Pxx prior to the recursive estimation. I might suggest starting with the same process noise Q used for the original tracking parameter. You then would change

```
Pxx(:,:,1)=blkdiag(Q,Q,R,R);
```

and after the recursive call to vossut, add

```
Pxx(1,1,k)=Q;
Pxx(2,2,k)=Q;
```

Don't forget to pass dq to the observation function, and also to the integrator (vossFNfct) within the vossut function where the sigma points are integrated in a vectorized fashion (a brilliant programming step by Voss). Within vossFNfct, strip out the parameters as a matrix p, pass them all to fc, and set up vectors within fc now. Don't forget to use .* and ./ for the final integration step in vossFNfct.

• Are the results resonable? Probably not! One of the unknown issues with such tracking is setting the relative process noises Q. We will find this again dramatically shown later in the book when dealing with Parkinson's disease models (chapter 10).

• Now try setting the relative Q's differently after each vossut call:

```
Pxx(1,1,k)=Q;
Pxx(2,2,k)=Q*0.1;
```

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

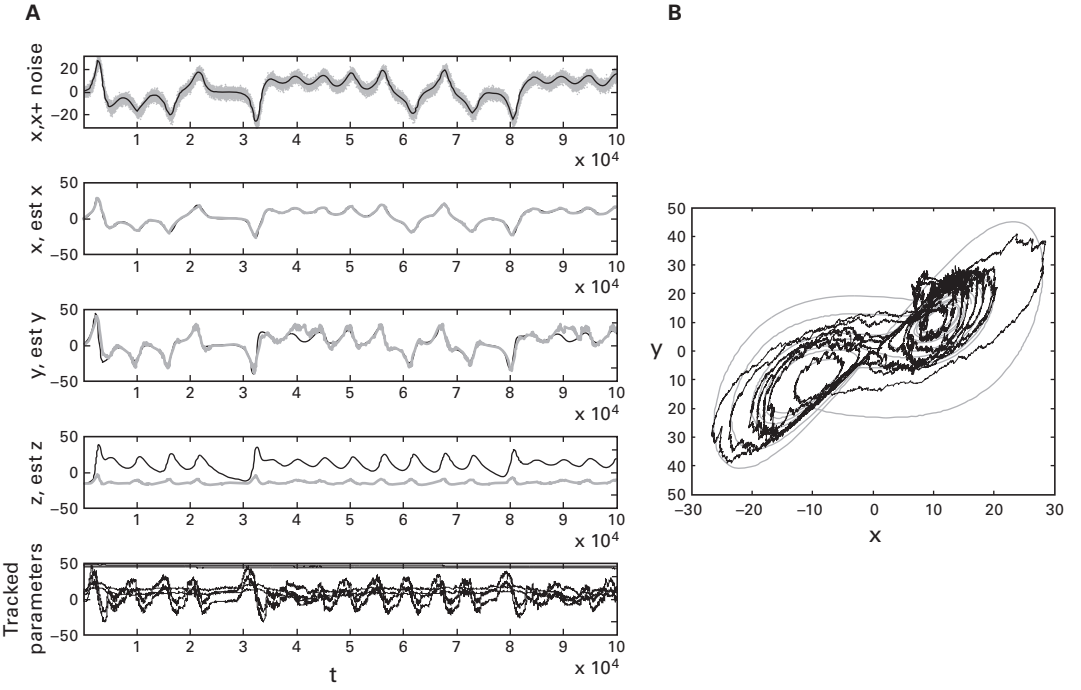


Figure 5.7 (plate 4)
Results of reconstructing the Lorenz equations using the unscented Kalman filter from measuring only noisy x variable and estimating variables and parameters. In this simulation, the integration time step is 0.0001, $\sigma = 10$, $r = 46$, and $b = 8/3$. A shows plots of reconstructed Lorenz variables. Upper panel is true (black) and noisy x variable (blue), second panel is true (black) and reconstructed x variable (red), third panel is true (black) and reconstructed y variable (red), fourth panel is true (black) and reconstructed z variable (red), and lower panel shows three reconstructed parameters with confidence limits. B shows true (black) and reconstructed (red) Lorenz x versus y trajectories. Note the difficulties with the z variable—similar troubles are seen in synchronizing Lorenz equations through this variable [PC90].

```
Pxx(3,3,k)=Q*0.1;
Pxx(4,4,k)=Q*0.1;
```

What happens to the tracking?

5.2. Now let's modify the algorithm from exercise 5.1 to track the Lorenz equations. There are three parameters and three variables to track.

• *Hint*: The Lorenz equations are very sensitive to initial conditions and thus round off error. So use a small integration time step—say, 0.001. Run the program for about 10,000 steps. Use the following to help get started:

```
Parameters [10; 46; 8/3]% sigma, r, and b
```

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

• Try using Q's of 0.015 like for the Fitzhugh-Nagumo equations. Examine at least the tracking of x and y, as well as the parameters. Can you track well?

• Now try using Q's proportional to the magnitude of the parameters, such as

```
Q1=10
Q2=46
Q3=8/3
```

Is this better?

• If you plot the true x variable against the estimated $\hat{x}$, you might get something that looks like figure 5.7 (plate 4), if you have done this well. Note in the figure that I have included the difficulties with the Lorenz z variable—not unexpected [PC90].

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026

Downloaded from direct.mit.edu/books/book/chapter-pdf/224779/9780262312080_cai.pdf by Eindhoven University of Technology user on 06 November 2026